Save as PDF Upgrade Share

explain the code as i am a 10 year old , but i still want to understand this deeply with code/* USER CODE BEGIN Header */
/**

```
******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
******************************************************************************
```

Show more

Absolutely. Think of this program as a **tiny security guard living inside your STM32F3-Discovery board**.

Its job is:

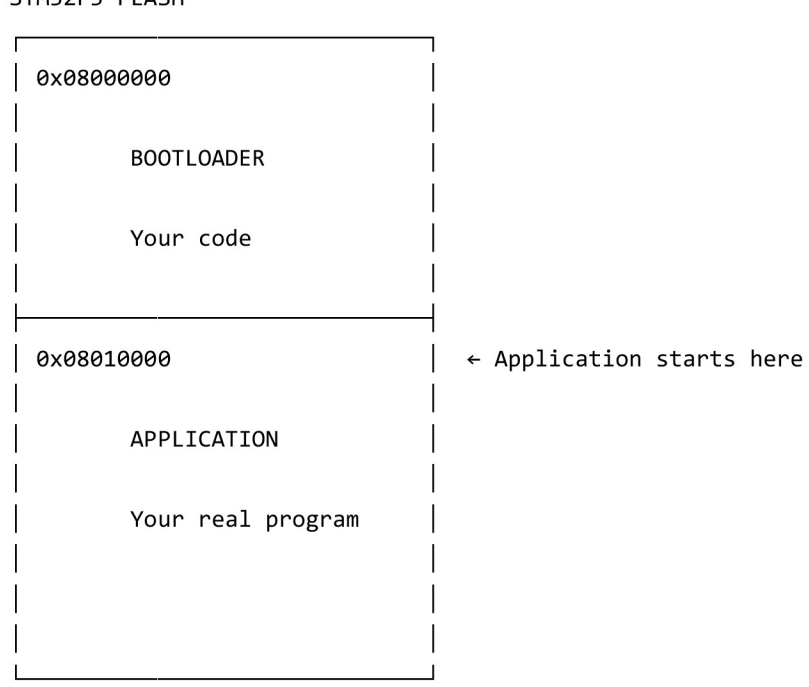
1. When the board starts, decide **"Should I run the normal application, or stay in bootloader mode?"**
2. If bootloader mode is selected, receive a new program through **USB**.
3. Erase the part of Flash where the application lives.
4. Write the received program into Flash.
5. Tell the PC `"OK\n"` .
6. Later, jump from the bootloader to the newly installed application.

I'll explain it from the ground up, but we'll gradually go from the **10-year-old picture → real microcontroller details → actual C code**.

1. First: What is this program?

Your STM32F3 has memory roughly like this:

Save as PDF



Your program deliberately creates **two programs inside Flash**:

Bootloader

Starts at:

```
0x08000000
```

Application

Starts at:

```
0x08010000
```

So your bootloader says:

> "I live at the beginning of Flash. The user's real program lives after me."

This is a very common embedded-systems design.

2. Why do we need a bootloader?

Imagine your normal application is:

Save as PDF

C

```c
int main(void)
{
    while (1)
    {
        LED_ON();
    }
}
```

Normally, to replace this program you might need:

```
PC
 │
 │ ST-LINK
 ▼
STM32
```

But your bootloader lets you do:

```
PC
 │
 │ USB
 ▼
STM32 Bootloader
 │
 │ writes Flash
 ▼
New Application
```

So the STM32 can essentially say:

"Give me a new program through USB and I'll install it myself."

That's the basic idea behind what you're building.

3. The most important memory idea

Before understanding the code, understand this:

C

```
0x08010000
```

Save as PDF

isn't a random number.

It is a **memory address**.

Think of memory like a giant apartment building.

```
Address          Room

0x08000000       Room 1
0x08000004       Room 2
0x08000008       Room 3
...
0x08010000       Application starts here
```

The STM32F3 Flash memory is mapped into the processor's address space.

So when you write:

C

```c
uint32_t app_address = 0x08010000;
```

you're saying:

"Remember the room where my application begins."

4. Your global variables

Let's start here:

C

```c
uint8_t usb_rx_buffer[2048];
uint32_t usb_rx_len = 0;
volatile uint8_t usb_rx_ready = 0;
```

These three are extremely important.

`usb_rx_buffer`

Save as PDF

C

```
uint8_t usb_rx_buffer[2048];
```

Imagine you have a box that can hold 2048 little pieces of information.

```
usb_rx_buffer

┌─────┬─────┬─────┬─────┬─────┬─────┬─────┬─ ... ─┐
│ 12  │ A4  │ 55  │ 9B  │ 32  │ 10  │       │       │
└─────┴─────┴─────┴─────┴─────┴─────┴─────┴───────┘
  0     1     2     3     4     5
```

Each element is:

C

```
uint8_t
```

A `uint8_t` is:

```
8 bits
= 1 byte
= 0 to 255
```

So:

C

```
uint8_t usb_rx_buffer[2048];
```

means:

Create a 2048-byte box for USB data.

5. What is `usb_rx_len`?

C

```
uint32_t usb_rx_len = 0;
```

Save as PDF

Suppose USB sends:

```
AA BB CC DD EE
```

Your buffer can hold 2048 bytes, but only 5 bytes arrived.

So:

```
buffer capacity = 2048
actual data     = 5
```

`usb_rx_len` tells you:

```
5
```

So:

C

```
usb_rx_len = 5;
```

means:

"There are 5 useful bytes inside the buffer."

6. What is `usb_rx_ready` ?

C

```
volatile uint8_t usb_rx_ready = 0;
```

This is like a little flag.

Initially:

```
usb_rx_ready = 0
```

meaning:

"No new USB data yet."

Save as PDF

When USB receives data, your USB callback changes it:

```
usb_rx_ready = 1
```

meaning:

"HEY! New data has arrived!"

Then your main loop sees:

C

```
if (usb_rx_ready == 1)
```

and processes the data.

7. Why `volatile`?

This is a very important embedded-C concept.

C

```
volatile uint8_t usb_rx_ready;
```

means roughly:

"Dear compiler, this variable can change unexpectedly. Don't assume its value stays the same."

Why?

Because your normal code might be doing:

C

```
while (1)
{
    if (usb_rx_ready == 1)
    {
        ...
    }
}
```

Save as PDF

But another piece of code, such as a USB interrupt/callback, can suddenly do:

C

```
usb_rx_ready = 1;
```

The compiler needs to understand that.

8. Your Flash variables

Next:

C

```
uint32_t flash_write_address = 0x08010000;
uint8_t is_flash_erased = 0;
```

The first one tells the bootloader:

"Where should I put the next 4 bytes?"

Initially:

```
flash_write_address
        ↓
0x08010000
```

After writing 4 bytes:

```
0x08010004
```

After another 4 bytes:

Save as PDF

```
0x08010008
```

And so on.

So it's basically your **Flash writing pointer**.

9. `is_flash_erased`

C

```c
uint8_t is_flash_erased = 0;
```

This is another flag.

```
0 = Flash hasn't been erased yet
1 = Flash has already been erased
```

Why do this?

Because you don't want to erase the application every time a USB packet arrives.

Imagine receiving:

```
Packet 1
Packet 2
Packet 3
Packet 4
...
```

You want:

Save as PDF

```
Packet 1
   ↓
ERASE FLASH

Packet 2
   ↓
WRITE

Packet 3
   ↓
WRITE

Packet 4
   ↓
WRITE
```

NOT:

```
Packet 1 → erase
Packet 2 → erase
Packet 3 → erase
Packet 4 → erase
```

10. The most interesting function: `JumpToApplication()`

Now we reach the heart of your bootloader.

C

```c
void JumpToApplication(void)
```

This means:

"Stop being the bootloader and start running the application."

But there is something very important you need to understand first.

11. An STM32 application has a special beginning

At:

Save as PDF

```
0x08010000
```

your application doesn't simply start with:

```c
main()
```

Instead, the beginning of the application contains something called the **vector table**.

Simplified:

```
0x08010000
┌──────────────────────────┐
│ Initial Stack Pointer    │
├──────────────────────────┤
│ Reset Handler Address    │
├──────────────────────────┤
│ NMI Handler Address      │
├──────────────────────────┤
│ HardFault Handler        │
├──────────────────────────┤
│ ...                      │
└──────────────────────────┘
```

The first two values are extremely important.

12. First value = Stack Pointer

At:

```c
*(__IO uint32_t*) app_address
```

you're reading the first 32-bit value at:

```
0x08010000
```

That should contain something like:

Save as PDF

```
0x2000xxxx
```

This is the application's initial **Main Stack Pointer**.

Think of the stack as the CPU's scratch/work table.

13. Second value = Reset Handler

This line:

C

```c
uint32_t jump_address = *(__IO uint32_t*) (app_address + 4);
```

reads:

```
0x08010004
```

because:

```
0x08010000 + 4
=
0x08010004
```

And that location contains the application's **Reset Handler address**.

For example:

```
0x08010231
```

So your bootloader is basically saying:

"Where does the application want me to start executing?"

14. Function pointer

Then:

C

Save as PDF

```
pFunction Jump = (pFunction) jump_address;
```

Earlier you created:

C

```
typedef void (*pFunction)(void);
```

This looks scary, but it's actually simple.

It means:

`pFunction` is a variable that can hold the address of a function that takes nothing and returns nothing.

Normally:

C

```
void Hello(void)
{
}
```

A function exists somewhere in memory.

For example:

```
Hello()
   ↓
0x08012345
```

A function pointer lets you store:

C

```
pFunction Jump = (pFunction)0x08012345;
```

Then:

C

```
Jump();
```

Save as PDF

means:

"Go execute the code at 0x08012345."

That's exactly what your bootloader is doing.

15. Disable interrupts

C

```
__disable_irq();
```

Imagine the bootloader is packing its suitcase and leaving the house.

You don't want somebody suddenly yelling:

"USB interrupt!"

while you're halfway through moving into the application.

So:

C

```
__disable_irq();
```

temporarily tells the CPU:

"Don't respond to interrupts right now."

16. Turn off USB interrupt

C

```
HAL_NVIC_DisableIRQ(USB_LP_CAN_RX0_IRQn);
```

The NVIC is basically the STM32's:

Interrupt manager

Save as PDF

You're telling it:

"Stop allowing this particular USB-related interrupt."

17. Turn off USB clock

C

```c
__HAL_RCC_USB_CLK_DISABLE();
```

The RCC controls clocks for peripherals.

Think of a clock as electricity saying:

```
Tick
Tick
Tick
Tick
```

The USB peripheral needs a clock to operate.

You're saying:

```
USB CLOCK → OFF
```

because you're leaving the bootloader.

18. Reset the clock system

C

```c
HAL_RCC_DeInit();
```

Your bootloader configured the clock system.

Your application will probably configure it again.

So you're saying: Save as PDF

> "Let's return the clock hardware to a clean/default state before handing control over."

This is important because otherwise the application may start with the bootloader's hardware configuration still active.

19. What is SysTick?

You then do:

C

```c
SysTick->CTRL = 0;
SysTick->LOAD = 0;
SysTick->VAL  = 0;
```

SysTick is a little timer inside the Cortex-M CPU.

STM32 HAL uses it for things like:

C

```c
HAL_Delay(100);
```

The timer creates periodic ticks.

So you're cleaning it up:

```
SysTick
   ↓
STOP
```

20. Deinitialize HAL

C

```c
HAL_DeInit();
```

This tells the HAL:

Save as PDF

"We're finished with the hardware setup done by this program."

Again, you're trying to leave the hardware in a clean state for the application.

21. Set the application's stack

This line is VERY important:

C

```
__set_MSP(*(__IO uint32_t*) app_address);
```

Remember:

```
0x08010000
     ↓
Initial Stack Pointer
```

So you're effectively doing:

```
CPU Stack Pointer
       ↓
Application's stack location
```

Now the CPU has the application's expected stack.

22. Finally...

C

```
Jump();
```

This is the big moment.

The bootloader says:

"My job is finished. Application, it's you Save as PDF

CPU execution jumps to the Reset Handler of your application.

Conceptually:

```
BOOTLOADER

        |
        | Jump()
        ▼

APPLICATION
        |
        ▼
Reset_Handler()
        |
        ▼
main()
```

23. Now let's look at `main()`

The program starts:

C

```c
int main(void)
{
```

Then:

C

```c
HAL_Init();
```

This initializes the STM32 HAL system.

Among other things, this prepares things needed by the HAL and initializes the SysTick timing mechanism.

24. Configure clocks

Save as PDF

C

```
SystemClock_Config();
```

This is where you tell the STM32:

"How fast should you run?"

Your configuration uses:

C

```
HSE
 ↓
PLL
 ↓
6×
 ↓
System Clock
```

Specifically:

C

```
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL6;
```

So you're taking the external clock and multiplying it by 6.

25. Configure GPIO

C

```
MX_GPIO_Init();
```

This sets up your pins.

For example:

C

Save as PDF

```
GPIO_InitStruct.Pin = B1_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
```

The blue USER button is configured as an input.

So the STM32 can ask:

C

```
HAL_GPIO_ReadPin(...)
```

"Is the button pressed?"

26. Configure I2C

C

```
MX_I2C1_Init();
```

I^2C is a communication protocol.

Think:

```
STM32
  │
  ├── SDA ──── sensor
  │
  └── SCL ──── sensor
```

Your code configures I2C1.

However, **your bootloader code shown here doesn't actually use I2C afterward**.

So this is probably generated by CubeMX because I2C1 was enabled in your project.

27. Configure SPI

C

Save as PDF

```
MX_SPI1_Init();
```

Same idea.

SPI is another communication protocol:

```
STM32
 │
 ├── SCK
 ├── MOSI
 ├── MISO
 └── CS
      │
      ▼
    Device
```

Again, the bootloader code shown doesn't actually use SPI.

28. Initialize USB

C

```
MX_USB_DEVICE_Init();
```

This is VERY important for your bootloader.

This starts the USB device system.

Your computer can now communicate with the STM32 through USB.

The CDC interface makes the STM32 appear somewhat like a serial/COM device.

29. Turn on red LED

C

```
HAL_GPIO_WritePin(GPIOE, GPIO_PIN_9, GPIO_PIN_SET);
```

You're setting PE9 HIGH.

Save as PDF

On your board this corresponds to the red LED you're using.

So:

```
PE9 → HIGH
      ↓
    RED LED ON
```

30. Wait 100 ms

C

```c
HAL_Delay(100);
```

This waits about:

```
100 milliseconds
=
0.1 second
```

Why?

To give the hardware a little time to settle.

31. The boot button decision

Now:

C

```c
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
```

This reads PA0.

But there's an important detail.

Your comment says:

Save as PDF

C

```
// Check if the Blue USER button (PA0) is pressed (HIGH)
```

but the actual condition checks:

C

```
GPIO_PIN_RESET
```

So the **comment and code disagree**.

Your code says:

"If PA0 is LOW, enter the bootloader decision branch."

And then:

C

```
JumpToApplication();
```

is called.

So according to the actual code:

```
PA0 == RESET
     ↓
Jump to application
```

If PA0 isn't RESET:

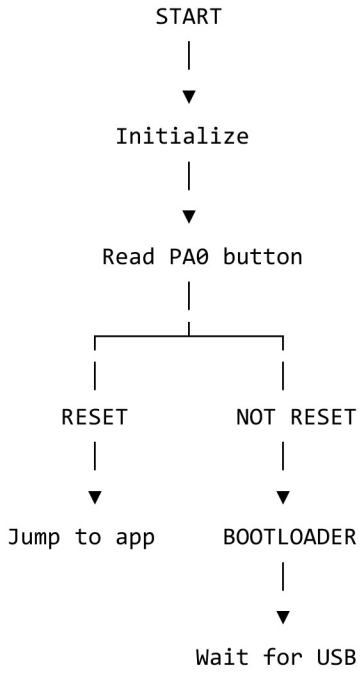
```
PA0 != RESET
     ↓
stay in bootloader
```

You should verify the STM32F3-Discovery button's actual electrical behavior rather than trusting that comment.

32. Your program therefore behaves like this

At startup:

Save as PDF



That's the main architecture.

33. Now the `while(1)` loop

If the bootloader remains active:

C

```
while (1)
{
```

This means:

"Do this forever."

Microcontrollers usually work this way.

Unlike a PC program that eventually finishes:

Save as PDF

```
start
 ↓
do something
 ↓
finish
```

embedded firmware often does:

```
start
 ↓
initialize
 ↓
┌──────────────┐
│              │
│ do things    │
│              │
└──────┬───────┘
       │
       └── repeat forever
```

34. Blink red LED

C

```
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_9);
HAL_Delay(100);
```

`Toggle` means:

```
OFF → ON
ON  → OFF
```

Every 100 ms.

So approximately:

Save as PDF

```
ON
OFF
ON
OFF
ON
OFF
```

This tells you:

> "The bootloader is running."

35. Check for USB data

Then:

C

```c
if (usb_rx_ready == 1)
```

Remember our flag?

USB callback probably does something like:

C

```c
usb_rx_ready = 1;
```

when data arrives.

So your main loop asks:

```
Did USB receive something?
        |
    ┌───┴────┐
    |        |
   NO       YES
    |        |
    ▼        ▼
 repeat   process it
```

36. Why copy the USB buffe

Save as PDF

You do:

```c
uint32_t len = usb_rx_len;
```

Then:

```c
uint8_t local_buf[2048];
```

Then:

```c
memcpy(local_buf, usb_rx_buffer, len);
```

This is a very interesting part.

Imagine USB is a delivery person.

The USB system is using:

```
usb_rx_buffer
```

Your Flash-writing code wants to work on the data.

Instead of both people fighting over the same box, you make a copy:

```
USB system
    |
    ▼
usb_rx_buffer
    |
    | memcpy()
    ▼
local_buf
    |
    ▼
Flash writer
```

So now the Flash writer works on its own c Save as PDF

37. What does `memcpy` do?

Suppose:

```
usb_rx_buffer:

AA BB CC DD EE
```

and:

C

```c
memcpy(local_buf, usb_rx_buffer, 5);
```

Afterwards:

```
local_buf:

AA BB CC DD EE
```

It's basically:

"Copy these bytes from here to there."

38. Then you clear the flag

C

```c
usb_rx_ready = 0;
```

Meaning:

"Okay, I received this packet. I don't need to process it again."

39. Erasing Flash

Save as PDF

Now:

C

```
if (is_flash_erased == 0)
```

First packet only.

Then:

C

```
HAL_FLASH_Unlock();
```

Flash is protected from accidental writing.

You have to unlock it before changing it.

Think:

FLASH 🔒

Then:

C

```
HAL_FLASH_Unlock();
```

becomes:

FLASH 🔓

40. Configure the erase

C

```
FLASH_EraseInitTypeDef EraseInitStruct;
```

Save as PDF

This creates a structure containing instructions for the Flash eraser.

Then:

C

```
EraseInitStruct.TypeErase = FLASH_TYPEERASE_PAGES;
```

You're saying:

"Erase Flash pages."

Then:

C

```
EraseInitStruct.PageAddress = 0x08010000;
```

Start erasing here.

Then:

C

```
EraseInitStruct.NbPages = 96;
```

Erase 96 pages.

Your comment says:

C

```
// 192 KB / 2 KB
```

because:

```
96 × 2 KB
=
192 KB
```

So your intended application region is:

Save as PDF

```
0x08010000
        ↓
192 KB
```

41. Why must Flash be erased?

Flash memory behaves differently from RAM.

RAM:

```
write 1
write 0
write 1
write 0
```

very easily.

Flash generally requires an erase operation before changing programmed bits back to the erased state.

Conceptually:

```
ERASED:

FF FF FF FF
```

Then you program:

```
12 34 56 78
```

So your bootloader first makes the application area clean.

42. Erase operation

C

```
if (HAL_FLASHEx_Erase(&EraseInitStruct, &PageError) != HAL_OK)
{
    Error_Handler();
}
```

Save as PDF

This says:

"Please erase the requested Flash pages."

If it fails:

C

```
Error_Handler();
```

43. Lock Flash again

C

```
HAL_FLASH_Lock();
```

After erasing:

```
FLASH → 🔒
```

This helps prevent accidental writes.

Then:

C

```
is_flash_erased = 1;
```

means:

"Okay, I've done the big erase."

44. Now comes an important problem: Flash writes 32 bits

Your code writes:

Save as PDF

C

```
FLASH_TYPEPROGRAM_WORD
```

A word here is:

```
32 bits
=
4 bytes
```

But USB data might not be a multiple of 4.

For example:

```
USB packet:

AA BB CC DD EE
```

That's:

```
5 bytes
```

But Flash programming wants:

```
4 bytes
```

at a time.

45. Calculate remainder

C

```
uint8_t remainder = len % 4;
```

Suppose:

```
len = 5
```

Then:

Save as PDF

```
5 % 4 = 1
```

So:

```
remainder = 1
```

We need 3 more bytes to reach 8.

46. Padding

Your code:

C

```c
if (remainder != 0) {
    for (uint8_t i = 0; i < (4 - remainder); i++) {
        local_buf[len + i] = 0xFF;
    }
    len += (4 - remainder);
}
```

For 5 bytes:

```
Original:

AA BB CC DD EE
```

Add:

```
FF FF FF
```

Now:

```
AA BB CC DD EE FF FF FF
```

That's 8 bytes.

Perfect.

Save as PDF

47. Why `0xFF` ?

Erased Flash normally reads as:

```
0xFF
```

So padding with:

```
0xFF
```

is a natural choice.

It basically says:

> "The remaining bytes aren't part of the real program."

48. Now unlock Flash again

C

```c
HAL_FLASH_Unlock();
```

because you're about to write.

Then:

C

```c
for (uint32_t i = 0; i < len; i += 4)
```

This is very important.

You're moving through the data 4 bytes at a time.

For:

```
8 bytes
```

you get:

Save as PDF

```
i = 0
i = 4
```

For:

```
12 bytes
```

you get:

```
i = 0
i = 4
i = 8
```

49. Building a 32-bit word

This code:

C

```c
uint32_t data_word = (local_buf[i])       |
                     (local_buf[i + 1] << 8)  |
                     (local_buf[i + 2] << 16) |
                     (local_buf[i + 3] << 24);
```

looks scary.

But let's break it down.

Suppose USB gives:

```
Byte 0 = 0x11
Byte 1 = 0x22
Byte 2 = 0x33
Byte 3 = 0x44
```

You want:

Save as PDF

```
0x44332211
```

because the STM32 is little-endian.

50. Why `<< 8`?

Take:

C

```
local_buf[i + 1]
```

Suppose it's:

```
0x22
```

Shift it left 8 bits:

C

```
0x22 << 8
```

becomes:

```
0x2200
```

Similarly:

```
0x33 << 16
=
0x330000
```

and:

```
0x44 << 24
=
0x44000000
```

Then `|` combines them:

Save as PDF

```
0x00000011
0x00002200
0x00330000
0x44000000
-----------
0x44332211
```

That's your 32-bit Flash word.

51. What is | ?

This:

C

```
|
```

is bitwise OR.

Think of it as putting pieces of a puzzle together.

```
00000011
00002200
00330000
44000000
```

No pieces overlap, so OR combines them into:

```
44332211
```

52. Write to Flash

Then:

C

```
HAL_FLASH_Program(
    FLASH_TYPEPROGRAM_WORD,
    flash_write_address,
    data_word
)
```

Save as PDF

You're saying:

> "STM32, please write this 32-bit number to this Flash address."

For example:

```
Address: 0x08010000

Data:
0x44332211
```

53. Move the Flash address

If writing succeeds:

C

```
flash_write_address += 4;
```

So:

```
Before:
0x08010000

After:
0x08010004
```

Next packet continues from there.

This is how the entire application gets assembled in Flash.

54. Imagine the PC sends 3 packets

Suppose:

Save as PDF

```
Packet 1 = 100 bytes
Packet 2 = 100 bytes
Packet 3 = 80 bytes
```

The bootloader does:

```
Packet 1
   ↓
Flash 0x08010000
        ↓
        100 bytes

Packet 2
   ↓
Flash 0x08010064
        ↓
        100 bytes

Packet 3
   ↓
Flash 0x080100C8
        ↓
        80 bytes
```

Eventually the complete `.bin` file is sitting in Flash.

55. Lock Flash

After writing:

C

```c
HAL_FLASH_Lock();
```

Again:

```
Flash → 🔒
```

56. Green LED

Save as PDF

C

```
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_11);
```

This toggles the green LED.

So you have:

```
RED LED
  ↓
Bootloader is alive

GREEN LED
  ↓
USB packet successfully processed
```

That's a nice debugging mechanism.

57. ACK

Then:

C

```
uint8_t ack[] = "OK\n";
```

creates:

```
'O'
'K'
'\n'
```

Then:

C

```
CDC_Transmit_FS(ack, sizeof(ack) - 1);
```

sends it back to the PC.

So the communication becomes:

Save as PDF

```
PC
 │
 │  DATA
 ▼
STM32
 │
 │ "OK\n"
 ▼
PC
```

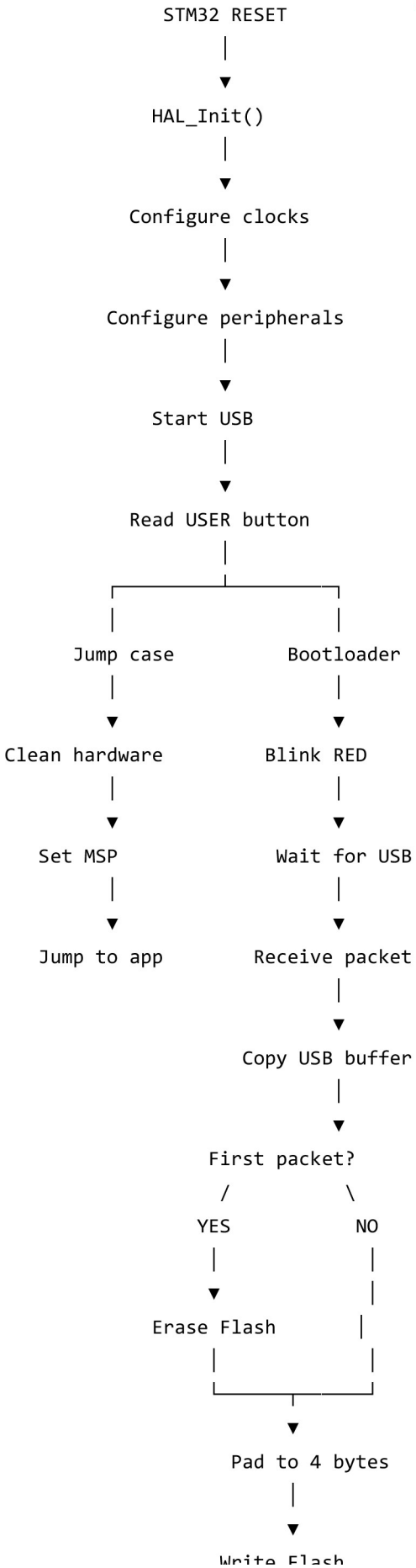
The PC can therefore know:

> "The STM32 received and processed this packet."

58. Your entire bootloader in one picture

This is the most useful mental model:

Save as PDF


STM32 RESET
HAL_Init()
Configure clocks
Configure peripherals
Start USB
Read USER button
Jump case
Bootloader
Clean hardware
Blink RED
Set MSP
Wait for USB
Jump to app
Receive packet
Copy USB buffer
First packet?
YES
NO
Erase Flash
Pad to 4 bytes
Write Flash

```
Write Flash
      |
      ▼
Green LED
      |
      ▼
   "OK\n"
      |
      ▼
  Wait again
```

Save as PDF

59. One REALLY important missing piece

There is a piece of your system that isn't shown in this `main.c`.

This:

C

```
usb_rx_ready
usb_rx_buffer
usb_rx_len
```

must be modified somewhere.

Most likely it's inside:

```
usbd_cdc_if.c
```

especially:

C

```
CDC_Receive_FS()
```

You probably have something conceptually like:

C

```
static int8_t CDC_Receive_FS(uint8_t* Buf, uint32_t *Len)
{
    memcpy(usb_rx_buffer, Buf, *Len);

    usb_rx_len = *Len;
    usb_rx_ready = 1;

    USBD_CDC_SetRxBuffer(&hUsbDeviceFS, &Buf[0]);
    USBD_CDC_ReceivePacket(&hUsbDeviceFS);

    return (USBD_OK);
}
```

Save as PDF

That is the **bridge between USB and your main loop**.

The flow is:

```
USB cable
   │
   ▼
USB hardware
   │
   ▼
USB interrupt
   │
   ▼
CDC_Receive_FS()
   │
   ├── copy data
   ├── save length
   └── usb_rx_ready = 1
            │
            ▼
        main while()
            │
            ▼
        write Flash
```

This is an extremely important concept to understand.

60. Interrupt vs `while(1)`

Here's the clever part.

Your CPU doesn't constantly sit there askin Save as PDF

C

```
"USB data yet?"
"USB data yet?"
"USB data yet?"
"USB data yet?"
```

Instead, USB hardware can interrupt the CPU.

Think of it like this:

Main program

```
CPU:
"Blink LED..."
"Blink LED..."
"Blink LED..."
```

Suddenly USB arrives

```
USB:
"HEY! DATA ARRIVED!"

CPU:
"Oh!"

CPU → USB interrupt
CPU → CDC_Receive_FS()
CPU → save data
CPU → return to main program
```

Then the main loop sees:

C

```
usb_rx_ready == 1
```

and processes it.

This is one of the most important ideas in embedded programming.

Save as PDF

61. Why your local buffer is useful

Imagine USB is receiving the next packet while you're writing Flash.

If both use:

C

```
usb_rx_buffer
```

you could have a race.

For example:

```
Flash writer:
reading byte 0
reading byte 1
reading byte 2
```

Meanwhile:

```
USB:
"Here is new data!"
```

and overwrites the same buffer.

Bad.

Your:

C

```
memcpy(local_buf, usb_rx_buffer, len);
```

creates a snapshot:

Save as PDF

```
USB BUFFER
    |
    | COPY
    ▼
LOCAL BUFFER
    |
    ▼
Flash writer
```

That's much safer.

62. Now let's understand the clock configuration

You have:

C

```c
RCC_OscInitStruct.HSEState = RCC_HSE_BYPASS;
```

This means the STM32 expects an external clock signal rather than necessarily using a normal crystal directly.

Then:

C

```c
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
```

means:

```
HSE
 ↓
PLL
```

Then:

C

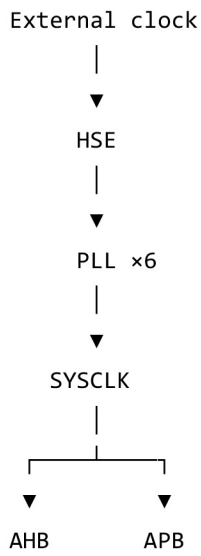
```c
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL6;
```

means:

Save as PDF

```
PLL output = HSE × 6
```

So the clock tree is approximately:



63. Why clocks matter

Almost everything inside an MCU depends on timing.

For example:

```
CPU
USB
UART
SPI
I2C
Timers
```

all need clocks.

If you configure the clock incorrectly:

```
USB communication ❌
UART baud rate ❌
timers ❌
CPU speed ❌
```

So clock configuration is one of the fundar Save as PDF nbedded programming.

64. Your I2C configuration

You have:

C

```
hi2c1.Instance = I2C1;
```

meaning:

Use the STM32's I2C1 peripheral.

Then:

C

```
hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
```

Most common I^2C devices use 7-bit addresses.

For example:

```
SSD1306 OLED
address = 0x3C
```

Then:

C

```
HAL_I2C_Init(&hi2c1);
```

actually applies the configuration to the hardware.

65. Your SPI configuration has something unusual

This line:

C Save as PDF

```
hspi1.Init.DataSize = SPI_DATASIZE_4BIT;
```

means SPI is configured for **4-bit data size**.

That's unusual for many normal SPI devices.

Typically you'd see:

C

```
SPI_DATASIZE_8BIT
```

for things like:

```
SD cards
OLEDs
Flash chips
sensors
LoRa modules
```

So if you're planning to actually use SPI1, this setting is worth checking.

66. GPIO initialization

You enable GPIO clocks:

C

```
__HAL_RCC_GPIOE_CLK_ENABLE();
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOF_CLK_ENABLE();
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOB_CLK_ENABLE();
```

Why?

Because GPIO peripherals also need clocks.

You can think of:

Save as PDF

```
GPIOE
```

as a machine.

Before using the machine:

```
Power/clock ON
```

Then configure its pins.

67. Output vs input

Your LED:

C

```
GPIO_MODE_OUTPUT_PP
```

means:

STM32 controls this pin.

So:

C

```
HIGH → LED state
LOW  → LED state
```

Your button:

C

```
GPIO_MODE_INPUT
```

means:

STM32 reads this pin.

So:

C

Save as PDF

```
HIGH / LOW
```

is information coming **into** the MCU.

68. `Error_Handler()`

Finally:

C

```c
void Error_Handler(void)
{
    __disable_irq();

    while (1)
    {
    }
}
```

If something catastrophic happens:

```
ERROR
  ↓
disable interrupts
  ↓
STOP FOREVER
```

This is called a **fail-stop** style.

For debugging, you might improve it by blinking an LED:

C

```
void Error_Handler(void)
{
    __disable_irq();

    while (1)
    {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_9);
        HAL_Delay(500);
    }
}
```

Save as PDF

Then:

```
Fast blink → bootloader

Slow blink → ERROR
```

That's often useful on embedded hardware.

69. The most important C concepts in your program

Your code is actually teaching you several advanced embedded-C concepts.

1. Arrays

C

```
uint8_t buffer[2048];
```

Means:

```
many bytes stored together
```

2. Pointers

C

```
uint32_t *ptr;
```

Means:

"This variable contains the address of sc Save as PDF

3. Dereferencing

C

```
*ptr
```

means:

"Go to that address and read what's there."

That's why:

C

```
*(__IO uint32_t*)0x08010000
```

means approximately:

"Treat 0x08010000 as an address containing a 32-bit value, then read that value."

4. Function pointers

C

```
pFunction Jump;
```

means:

"Jump can hold the address of a function."

Then:

C

```
Jump();
```

executes it.

Save as PDF

5. `volatile`

C

```
volatile uint8_t usb_rx_ready;
```

means:

"This value can change outside the normal flow of this code."

Very common with interrupts and hardware.

6. Bit shifting

C

```
value << 8
```

moves bits to the left.

Used here to construct a 32-bit Flash word.

7. Bitwise OR

C

```
a | b
```

combines individual bits.

8. Structures

C

```
FLASH_EraseInitTypeDef EraseInitStruct;
```

A structure groups related information tog Save as PDF

Think:

```
EraseInitStruct
 ├── TypeErase
 ├── PageAddress
 └── NbPages
```

70. The whole program in "10-year-old language"

Imagine the STM32 is a little robot.

It has two programs stored inside its brain:

```
BOOTLOADER
APPLICATION
```

When the robot wakes up:

```
Robot wakes
   ↓
Check button
```

If the button says:

```
"Run normally!"
```

the robot:

```
cleans itself
   ↓
finds application
   ↓
jumps into application
```

Otherwise:

```
stay in bootloader
```

Then it waits for the computer:

Save as PDF

```
PC:
"Here's a piece of your new program."

STM32:
"Okay!"

STM32:
"Erase old program."

STM32:
"Write this piece."

STM32:
"Done."

STM32:
"OK"
```

Then:

```
PC:
"Here's another piece."

STM32:
"Okay!"

...
```

Eventually:

```
NEW APPLICATION
        ↓
0x08010000
        ↓
ready to run
```

Then the next time the board starts, the bootloader can jump into it.

71. One important thing to fix/verify

Save as PDF

There are a few things I would **not blindly assume are correct** in this code.

Button comment vs actual logic

You have:

C

```
// Button NOT pressed -> jump to the application
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
{
    JumpToApplication();
}
```

The comment says one thing, but the condition depends on whether the button is active-high or active-low.

SPI is configured as 4-bit

C

```
hspi1.Init.DataSize = SPI_DATASIZE_4BIT;
```

That is unusual and should be intentional.

Application vector table

For this to work correctly, your **application project must actually be linked to**:

```
0x08010000
```

and its vector table must also be relocated appropriately.

Otherwise the bootloader can correctly jump to the application's Reset Handler, but interrupts inside the application may still point to the bootloader's vector table.

That's a very important bootloader concep Save as PDF

72. The next thing you should learn

Since you're building this as a **real custom STM32 bootloader**, I would learn it in this order:

```
STEP 1
Memory addresses
      ↓
STEP 2
Pointers and dereferencing
      ↓
STEP 3
STM32 Flash
      ↓
STEP 4
Vector table
      ↓
STEP 5
Stack Pointer / MSP
      ↓
STEP 6
Function pointers
      ↓
STEP 7
Interrupts
      ↓
STEP 8
USB CDC
      ↓
STEP 9
Bootloader → Application jump
      ↓
STEP 10
PC sends .bin file
      ↓
STEP 11
CRC/checksum
      ↓
STEP 12
Reliable firmware update
```

The **next big concept** I recommend unders Save as PDF **ct contents of** `0x08010000`, `0x08010004`, **the vector table, MSP, Reset_Handler, and why** `SCB->VTOR` **usually matters when jumping to an application**. Once you understand those, the `JumpToApplication()` function will stop looking like "magic code" and you'll understand exactly what the Cortex-M4 is doing at the CPU level.

ChatGPT can make mistakes. Check important info.